

[← Go Back](#)

Hardware

Due



Tutorials

[Extended SPI Library Usage with the Arduino Due](#)[Due Motor Shield Example](#)[Arduino Due Keyboard Controller](#)[Mouse Controller](#)[Due Multiple Blinks Example](#)[Simple Waveform Generator with Arduino Due](#)[Keyboard Controller](#)[Upgrading Firmware of the Arduino Due Programming Port 16U2](#)

Home / Hardware / Due / **Simple Waveform Generator with Arduino Due**

Simple Waveform Generator with Arduino Due

Generate waveforms by using the Arduino Due and its DAC features

Author • Arduino Last revision • 05/29/2023

ON THIS PAGE

Introduction

[Goals](#)[Hardware & Software Needed](#)[Circuit](#)[Programming the Board](#) +[Testing It Out](#) +[Conclusion](#)

Introduction

This tutorial shows how to make a simple waveform generator by using the DAC features of the Arduino Due board.

With push buttons, you will be able to choose a waveform shape (sine, triangular, sawtooth, or square) that we will send to to send to the DAC0 and DAC1 channels and change the frequency of the generated signal with a potentiometer.

Goals

Create a simple waveform generator.

Learn about DAC features on the DUE board.

Hardware & Software Needed

[Help](#)

10K Ω potentiometer
2x push buttons
2x 10K Ω resistors
Jumper wires
Breadboard
Arduino IDE ([online](#) or [offline](#)).
[Waveforms library](#)
Oscilloscope (optional)

Circuit

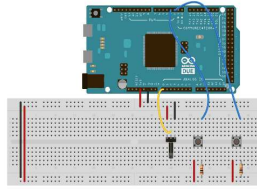
Connect power and ground on your breadboard to the Arduino. In the image below, the red (power) and black (ground) wires connect to the two long vertical rows on the breadboard, providing access to 3.3V and ground.

Connect a wire from digital pin 2 to one leg of a pushbutton. That same leg of the button connects through a pull-down resistor (10-kilohm) to ground. The other leg of the button connects to the 3.3V power.

Wire up another button in the same fashion, but to digital pin 3.

Hook up the potentiometer by connecting one side to power and the other side to ground. The pin in the middle of the potentiometer goes to analog input 0.

Pins DAC0 and DAC1 will generate the waveform. You can use an oscilloscope to visualize the generated waveforms.



The circuit for this tutorial.

Programming the Board

The waveforms are stored inside a two-dimensional array where each row represent a different waveform shape. The waveform samples are contained inside the columns, so you can access the waveform table using two indexes:

```
1 waveformsTable[waveform
```

With the `waveformIndex` array, you choose which samples to read. By incrementing the `sampleIndex` array from 0 to the maximum in a fixed time, you will create the waveform shape. Repeating this procedure continuously and sending the samples values on the DAC output will give you a constant signal.

In order to choose the waveform shape with a push button, match the button press to the `waveformIndex` increment. You can use the interrupts, triggering

when the Arduino Due sees a rising edge on the button pin, it will execute the function linked to the interrupt matched with the button:

```
void waveCh0_select() and  
void waveCh1_select()
```

The potentiometer connected to analog pin 0 is used to choose the sample rate and the period of the signal is given by the sample rate multiplied for the number of the samples.

Taking into account the time for the instructions to execute, and adding the time for the analog input (around 40 μ S to read the pot), maximum frequency for the signal with this sketch is around 170 Hz.

The sketch is composed of two files. One has the two-dimensional arrays, with the table of the samples for all the waveforms for legibility. Download the attached file from [GitHub](#), or if you want to start from scratch you have to create a new folder inside your sketchbook folder and place the two files inside. The sketch file must have the same name of the folder, and the file with the sample table must be named `"Waveforms.h"`.

Code

FunctionGenerator.ino

```
1  /*
2   Simple Waveform gen
3
4   * connect two push
5   with a 10 kilohm
6   to send to the DA
7   * connect a 10 kilo
8   signal frequency
9
10  */
11
12  #include "Waveforms.h"
13
14  #define oneHzSample 1
15
16  const int button0 = 2
17  volatile int wave0 =
18
19  int i = 0;
20  int sample;
21
22
23  void setup() {
24    analogWriteResoluti
25    analogReadResolutio
26
27    attachInterrupt(but
28    attachInterrupt(but
```

Waveforms.h

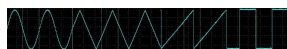
```

1  #ifndef _Waveforms_h_
2  #define _Waveforms_h_
3
4  #define maxWaveform 4
5  #define maxSamplesNun
6
7  static int waveformsT
8  // Sin wave
9  {
10     0x7ff, 0x86a, 0x8
11     0xbff, 0xc5a, 0xc
12     0xeec, 0xf1f, 0xf
13     0xfff, 0xffc, 0xf
14     0xeec, 0xeb4, 0xe
15     0xbff, 0xba1, 0xb
16     0x7ff, 0x794, 0x7
17     0x3ff, 0x3a4, 0x3
18     0x112, 0xdf, 0xb1
19     0x0, 0x2, 0xb, 0x
20     0x112, 0x14a, 0x1
21     0x3ff, 0x45d, 0x4
22 }
23 ,
24
25 // Triangular wave
26 {
27     0x44, 0x88, 0xcc,
28     0x2ec, 0x330, 0x3

```

Testing It Out

After you have uploaded the code, use the pushbuttons that you have hooked up to the breadboard to generate waveforms. If you have an oscilloscope, you can use it to visualize the different waveforms that you generate. You can control the signal frequency by using the potentiometer connected to `A0`.



Waveforms shown on
an oscilloscope

Troubleshoot

If the code is not working, there are some common issues we can troubleshoot:

- The wiring is incorrect.
- You have not installed or included all of the libraries.
- Make sure there are no missing curly brackets {}.
- We have not selected the right port and board.

Conclusion

In this example, we have learned how to generate waveforms by using the DAC features of the Arduino Due.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

